

DevCore AI — File Structure & Architecture Guide

Complete Technical Documentation, File Purpose Breakdown, & Path Connections Map

1. Executive Overview & Core Technology Stack

DevCore AI is an autonomous multi-agent software engineering operating system. Built with **Python** and **Streamlit**, it orchestrates a collaborative AI software development team (Product Manager, Solution Architect, Tech Lead, Code Generator, QA/Tester, Security Specialist) using **LangGraph** state graphs, **SQLite** database persistence, **Groq Cloud & Local Ollama LLMs**, and a **ChromaDB RAG** vector engine.

2. Complete Workspace Directory & File Structure

```
project 2 multi ai/
■■■■ .env # Environment variables & API configuration
■■■■ Dockerfile # Container build definition
■■■■ docker-compose.yml # Container deployment composition
■■■■ requirements.txt # Python dependency manifest
■■■■ app.py # Main Streamlit SPA Entrypoint & Router Hub
■■■■ theme.py # Design system palette definitions & color tokens
■■■■ styles.py # CSS binding engine for Streamlit injection
■■■■ styles.css # Core stylesheet (Variables, Dark mode, Cards, Widget
s)
■■■■ ai_software_company.db # SQLite database (Projects, Blueprints, Chat, Logs)
■■■■ pagepad.db # Auxiliary DB for UI state and notes
■■■■ .streamlit/config.toml # Streamlit server config
■■■■ agents/ # AI Agent Engine & Persona Definitions
■ ■■■■ base.py # Agent execution core (LLM dispatcher, parsing)
■ ■■■■ prompts.py # System instructions for all 6 agent roles
■■■■ components/ # Reusable UI Components & Modals
■ ■■■■ cards.py # Status, metric, and project cards
■ ■■■■ groq_quota_modal.py # Groq API quota limit detection & fallback modal
■ ■■■■ implementation_studio.py # Live Code Studio (Multi-file editor & preview)
■ ■■■■ navigation.py # Top Navbar, logo, theme selector, navigation
■ ■■■■ ui.py # Custom HTML badges, progress indicators, alerts
■ ■■■■ chatbot/ # Copilot Floating Chatbot Subsystem
■ ■■■■ chat_window.py # Main chat thread dialog window
■ ■■■■ floating_chat.py # Floating action button (FAB) widget
■ ■■■■ groq_client.py # Groq API wrapper client
■ ■■■■ message_renderer.py # HTML chat bubble renderer & syntax highlighting
■ ■■■■ ollama_client.py # Local Ollama client interface
■ ■■■■ prompt_templates.py # Copilot system prompts
■■■■ database/ # Database Layer
■ ■■■■ connection.py # SQLite query helpers & transaction locks
■ ■■■■ schema.py # Database tables creation (Projects, Blueprints, Log
s)
■■■■ exports/ # Document & PDF Export Engine
■ ■■■■ generate_master_pdf.py # Generates full Master Blueprint PDF
■ ■■■■ generate_sla_pdf.py # Generates SLA & Compliance PDF
■ ■■■■ markdown.py # Exports project blueprints to Markdown
■ ■■■■ pdf_generator.py # Core ReportLab PDF styling engine
■■■■ icons/lucide.py # Lucide SVG icon rendering helpers
■■■■ pages/ # Single Page Application (SPA) Views
■ ■■■■ dashboard.py # Dashboard View (Analytics, timeline, quick start)
■ ■■■■ new_project.py # Project Creator Wizard
■ ■■■■ projects.py # Project Workspace & LangGraph execution launcher
■ ■■■■ ai_team.py # AI Team Management & agent prompt testing
■ ■■■■ chat.py # Dedicated Full-Screen Team Chat interface
■ ■■■■ settings.py # Platform Configuration (API Keys, Theme, DB reset)
■■■■ rag/ # RAG Knowledge Engine
■ ■■■■ document_loader.py # Ingests PDF, TXT, MD, Python documents
■ ■■■■ vector_store.py # ChromaDB vector store & similarity retriever
■■■■ styles/ # Extra Style Assets
■ ■■■■ chatbot.css # Chatbot widget styles
■ ■■■■ neobrutalism.css # Retro-modern Neobrutalist design overrides
■■■■ utils/ # Utility Modules & Execution Engines
■ ■■■■ blueprint_context.py # Formats database blueprints for agent prompts
■ ■■■■ config.py # Environment settings & Ollama health check
■ ■■■■ consultation_engine.py # Multi-agent interactive consultation workflow
■ ■■■■ implementation_engine.py # Code generation engine & placeholder cleaner
■ ■■■■ ollama_client.py # REST client for local Ollama server
```

■ ■■■ prompt_builder.py	# Assembles strict system prompts for agents
■ ■■■ telemetry.py	# Performance tracking & response caching
■ ■■■ validator.py	# Code syntax checker & placeholder detector
■■■ workflow/	# LangGraph Agent Orchestration Engine
■■■ graph.py	# StateGraph DAG defining agent execution flow
■■■ state.py	# TypedDict defining shared project state schema

3. Detailed File Breakdown & Responsibilities

- **app.py**: Main SPA Entrypoint. Initializes database, injects CSS, renders Navbar, routes pages (Dashboard, New Project, Projects, AI Team, Chat, Settings), handles global errors, and mounts floating chatbot.
- **theme.py**: Theme dictionary defining color variables for Dark, Light, Cyberpunk, Neobrutalism, Midnight Blue, and Glassmorphism modes.
- **styles.py**: Python engine (`inject_design_system_css`) combining `theme.py` tokens with `styles.css` into Streamlit.
- **styles.css**: Master stylesheet controlling CSS variables, neobrutalist buttons, card components, dark theme rules, and responsive layouts.
- **agents/base.py**: Core agent execution engine (`run_agent`). Selects Groq/Ollama LLM provider, formats prompt context, parses structured JSON, and logs agent execution to SQLite.
- **agents/prompts.py**: System instructions and role specifications for all 6 AI personas (Product Manager, Architect, Tech Lead, Coder, QA, Security).
- **components/navigation.py**: Top Navigation Bar. Handles page state switching, logo display, quick actions, and theme selector dropdown.
- **components/implementation_studio.py**: Embedded Live Code Studio. Renders multi-file tabs, live code generation, execution logs, code editing, and file exporting.
- **components/groq_quota_modal.py**: Monitors Groq API rate limits and automatically displays fallback modal to switch to local Ollama models.
- **components/chatbot/chat_window.py**: Main copilot chat window. Processes user queries, retrieves vector RAG context, and displays agent responses.
- **components/chatbot/groq_client.py**: Fast streaming API wrapper for Groq Cloud LLM models (`llama-3.3-70b-versatile`).
- **components/chatbot/ollama_client.py**: Local REST API client for querying offline Ollama models (`deepseek-r1`, `llama3`).
- **database/schema.py**: Database initializer (`init_db`). Creates tables: `projects`, `blueprints`, `agent_logs`, `chat_history`, `project_memory`, `app_settings`.
- **database/connection.py**: SQLite helper (`execute_query`, `execute_update`) ensuring thread-safe database operations.
- **pages/dashboard.py**: Dashboard view displaying key metrics, recent projects, timeline logs, and system health status.
- **pages/new_project.py**: Project Creation Wizard. Captures title, domain, vision, architecture choice, and team configuration.
- **pages/projects.py**: Project Management Hub. Triggers LangGraph multi-agent pipeline, displays architectural blueprints, and opens Implementation Studio.
- **pages/ai_team.py**: AI Team Center. Customizes agent prompts, inspects roles, and runs single-agent tests in isolated sandboxes.

- **pages/chat.py:** Full-screen dedicated chat view for communicating with individual agents or full team with RAG search.
- **pages/settings.py:** Platform Settings. Manages Groq API keys, Ollama server URL, active UI theme, and database maintenance.
- **workflow/graph.py:** LangGraph StateGraph DAG orchestrating sequential execution: PM -> Architect -> Tech Lead -> Coder -> QA/Security.
- **workflow/state.py:** TypedDict schema (`ProjectState`) defining shared state passed through the LangGraph workflow.
- **rag/vector_store.py:** ChromaDB vector database manager for storing and querying project documentation embeddings.
- **rag/document_loader.py:** Document ingestion module parsing PDFs, TXT, Markdown, and code files into vector text chunks.
- **exports/generate_master_pdf.py:** Compiles full project PRD, architecture diagrams, and code into downloadable PDF blueprints.
- **utils/implementation_engine.py:** Code generation engine parsing multi-file code blocks and eliminating placeholder comments.
- **utils/config.py:** Environment variables loader & Ollama server connection health-checker (`ensure\_ollama\_server\_online`).

4. Data Flow & Path Connections Map

1. **Initialization:** app.py -> calls database/schema.py (init_db()) -> loads active theme from database/connection.py -> injects CSS via styles.py.
2. **Navigation & Layout:** app.py renders top bar via components/navigation.py & mounts floating chatbot via components/chatbot/floating_chat.py.
3. **Project Creation:** User creates project in pages/new_project.py -> saves metadata into ai_software_company.db.
4. **LangGraph Execution:** pages/projects.py launches workflow/graph.py -> executes state graph across agents/base.py using prompts in agents/prompts.py.
5. **LLM Dispatch:** agents/base.py routes queries to components/chatbot/groq_client.py (Groq Cloud) or utils/ollama_client.py (Local Ollama). If Groq rate limits, components/groq_quota_modal.py switches provider.
6. **Implementation Studio:** components/implementation_studio.py invokes utils/implementation_engine.py to generate multi-file code, validate with utils/validator.py, and display live tabs.
7. **RAG Knowledge Search:** User questions in pages/chat.py or floating chatbot pass to rag/vector_store.py to fetch relevant context from ingested documents.
8. **Export Engine:** pages/projects.py triggers exports/generate_master_pdf.py & exports/markdown.py to download completed project handoffs.

5. Key Architectural Insights for Developers

- **Custom SPA Routing:** Uses session state (st.session_state.current_page) to achieve instant page transitions without full page reloads.
- **Dual Model Fallback:** Supports high-speed cloud execution via Groq API with auto-fallback to privacy-first local Ollama LLMs when offline or rate-limited.
- **LangGraph State Machine:** Multi-agent orchestration is managed cleanly using a functional state graph, making role additions or branch conditions simple to maintain.

- **Database Auto-Migration:** SQLite database schema automatically self-heals and creates missing tables upon application startup.